

motores-de-busqueda

- Clase 05 — Motores de búsqueda, relevancia y recuperación aumentada
 - 1. Contexto: encontrar no es consultar una tabla
 - 2. Information Retrieval: recuperar documentos relevantes
 - 3. Recuperar, filtrar y rankear
 - 4. Experiencia de búsqueda en un catálogo
 - 5. El índice como read model derivado
 - 6. Escala, disponibilidad y límites operativos
 - 7. Búsqueda semántica, híbrida y RAG
 - 8. Casos reales
 - 9. Caso aplicado: búsqueda híbrida para una plataforma de venta
 - 10. Trade-offs y decisiones arquitectónicas
 - 11. Puntos clave
 - 12. Referencias

Clase 05 — Motores de búsqueda, relevancia y recuperación aumentada

ARQ 2026 · UCC · 9 de septiembre de 2026

1. Contexto: encontrar no es consultar una tabla

Una persona abre una plataforma de venta y escribe `rock en Buenos Aires`. No está pidiendo un registro cuyo identificador conoce: quiere **descubrir** opciones pertinentes. Espera que aparezcan eventos de rock aunque la descripción diga «recital», que se respeten fecha y zona, que los resultados disponibles tengan prioridad y que el orden resulte razonable.

En el sistema monolítico de Desarrollo de Software, una primera aproximación puede ser una consulta como esta:

```
SELECT *
FROM eventos
WHERE titulo LIKE '%rock%'
      AND ciudad = 'Buenos Aires';
```

Es válida para un catálogo pequeño y una necesidad simple, pero tiene límites importantes:

- Busca una cadena literal. Puede no encontrar «recital de rock» si la consulta tiene variaciones, errores de tipeo, acentos o sinónimos.
- No expresa qué resultado debe aparecer antes. Todos los registros que coinciden son equivalentes para SQL salvo que se agregue un criterio artificial de orden.
- Puede exigir recorrer muchos datos o usar índices que no están diseñados para análisis lingüístico y ranking.
- Mezcla la carga de descubrimiento, que suele ser alta e impredecible, con la base que respalda operaciones críticas como publicar, cobrar o reservar.

Un motor de búsqueda no reemplaza a una base relacional ni vuelve obsoleta una API de Catálogo. Cumple otra responsabilidad: construir una vista optimizada para **recuperar candidatos y ordenarlos por relevancia**. La base transaccional sigue siendo la fuente de verdad para modificar un evento, calcular el precio final o confirmar disponibilidad.

Esta distinción conduce a dos preguntas arquitectónicas:

1. ¿Cómo se recuperan y ordenan documentos para que la búsqueda sea útil?
2. ¿Cómo se mantiene ese índice alineado, aunque sea de forma eventual, con los servicios que son dueños de los datos?

1.1 Cuándo hace falta un motor especializado

No todo sistema necesita Elasticsearch, Solr u OpenSearch. Agregar un motor especializado crea un nuevo dato derivado, una ruta de sincronización y una dependencia operativa. Conviene evaluarlo cuando se combinan varias de estas condiciones:

- El catálogo contiene mucho texto libre o descripciones heterogéneas.
- La persona usuaria formula consultas abiertas, no solo búsquedas por identificador.
- Es necesario ordenar por relevancia y no solo filtrar.
- Hay filtros, facetas, sugerencias o corrección de typos que forman parte de la experiencia.
- La búsqueda recibe una carga que no debería competir con operaciones transaccionales.
- Encontrar un resultado tiene impacto de producto o negocio medible.

Para una tabla pequeña, consultas exactas y baja frecuencia, los índices y capacidades *full-text* de la base existente pueden ser suficientes. La decisión no es «SQL contra motor de búsqueda»: es elegir la complejidad mínima que satisface el problema.

2. Information Retrieval: recuperar documentos relevantes

Information Retrieval (IR) estudia cómo representar, organizar y recuperar documentos de una colección según una consulta. En una consulta por clave, la pregunta es «¿cuál es el registro `evt-123` ?». En IR, la pregunta es «¿qué documentos satisfacen mejor esta necesidad?». La segunda pregunta admite grados: un resultado puede ser más o menos relevante.

Un motor suele modelar cada elemento indexado como un **documento** con **campos**. Para un evento, un documento podría contener:

```
{
  "id": "evt-123",
  "titulo": "Festival de rock al aire libre",
  "descripcion": "Bandas locales e internacionales",
  "ciudad": "Buenos Aires",
  "fecha": "2026-10-18",
  "precio_desde": 18000,
  "disponible": true
}
```

Los campos no tienen todos el mismo uso. `titulo` y `descripcion` pueden participar en la recuperación textual; `fecha` y `precio_desde` son útiles para filtros; `disponible` puede ser un filtro o una señal de ranking. Diseñar el documento significa decidir qué lectura se quiere servir, no copiar sin criterio las tablas del servicio fuente.

2.1 Precision y recall

Para hablar de calidad se necesitan dos métricas básicas:

- **precision:** proporción de resultados devueltos que son relevantes.
- **recall:** proporción de todos los resultados relevantes que efectivamente fueron recuperados.

Supongamos que para una consulta existen diez eventos verdaderamente relevantes. Si el buscador devuelve cinco, de los cuales cuatro son pertinentes:

```
precision = 4 resultados relevantes devueltos / 5 resultados devueltos = 0,80
recall    = 4 resultados relevantes devueltos / 10 resultados relevantes = 0,40
```

Una consulta muy restrictiva puede mejorar *precision* y perder *recall*: devuelve pocos resultados muy buenos, pero deja afuera alternativas pertinentes. Una expansión con sinónimos puede mejorar *recall* y, a la vez, introducir ruido. No existe un valor correcto universal; el equilibrio depende del dominio y del daño relativo de mostrar algo irrelevante frente a ocultar algo útil.

Estas métricas requieren un conjunto de consultas y documentos evaluados por personas o reglas de negocio. No son iguales a métricas de producto como CTR o conversión. Un resultado puede recibir clics por su imagen o precio sin ser el mejor resultado semántico; y un resultado relevante puede no recibir clics por problemas de presentación. Se complementan, pero responden preguntas diferentes.

2.2 Análisis de texto

Antes de construir el índice, el motor analiza los campos de texto. El proceso suele incluir:

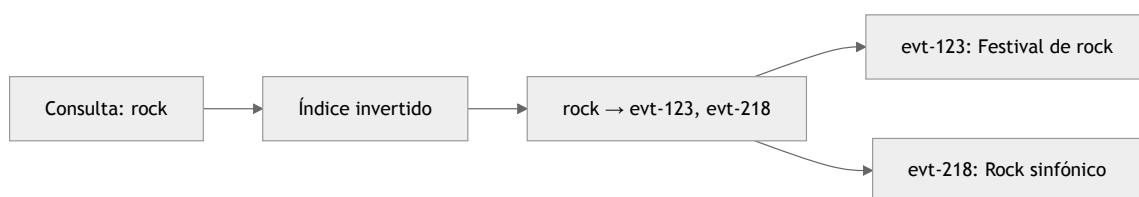
1. **Tokenización:** dividir un texto en unidades o términos. `Rock en Buenos Aires` puede producir `rock`, `en`, `buenos`, `aires`.

2. **Normalización:** llevar mayúsculas, acentos u otras variantes a una forma comparable. Así `Córdoba` y `cordoba` pueden coincidir si la política del dominio lo requiere.
3. **Stopwords:** tratar palabras muy frecuentes, como «en» o «de», que rara vez ayudan a distinguir documentos. No se eliminan ciegamente: en algunos dominios una palabra aparentemente común tiene significado.
4. **Stemming o lematización:** reducir variantes a una raíz aproximada o a una forma canónica. Según el analizador, «comprar», «compra» y «comprando» pueden aproximarse entre sí.

El mismo análisis, o uno compatible, se aplica a la consulta. Si se indexa `rock` en minúscula pero se compara literalmente con `Rock`, se pierde una coincidencia esperable. Elegir analizadores es una decisión dependiente de idioma, campo y vocabulario: un título, un SKU y una descripción extensa no necesariamente se tratan igual.

2.3 El índice invertido

La estructura central de la búsqueda léxica es el **índice invertido**. En vez de guardar solo documento → texto, se mantiene término → documentos que lo contienen.



Es parecido al índice al final de un libro: para encontrar dónde aparece un término no se leen todas las páginas. Las listas asociadas a cada término se conocen como *posting lists*. Además de identificar documentos, suelen guardar información para el ranking, por ejemplo cuántas veces aparece el término y en qué campo.

El índice invertido acelera la recuperación de candidatos, pero no decide por sí solo el orden final. Para la consulta `rock buenos aires`, el motor puede intersectar o combinar las listas de los términos y luego calcular una puntuación para los candidatos.

2.4 Modelo booleano como contraste

El modelo booleano usa operadores como `AND`, `OR` y `NOT`:

```
(rock OR recital) AND "buenos aires" NOT cancelado
```

Es útil para expresar restricciones y, en motores actuales, forma parte de consultas compuestas. Su límite es que trata la coincidencia como binaria: un documento satisface o no satisface la condición. No explica por qué un evento con «rock» en el título debería aparecer antes que otro que menciona la palabra una vez al final de una descripción extensa.

3. Recuperar, filtrar y rankear

Una búsqueda de catálogo no es una sola operación. Separar sus etapas evita decisiones confusas:



- **Análisis y expansión:** normaliza, corrige o incorpora sinónimos a la consulta. `recital` puede ayudar a recuperar documentos asociados a `rock`, según la política definida.
- **Recuperación:** obtiene un conjunto razonable de candidatos mediante índice invertido, vectores o ambos.
- **Filtros:** descartan candidatos que no cumplen una condición obligatoria, como una fecha, una ciudad o una categoría solicitada.
- **Ranking o reranking:** ordena los candidatos restantes mediante señales de relevancia textual, semántica y negocio.

Un sinónimo no es un filtro y la distancia geográfica no es necesariamente un filtro. Confundirlos lleva a diseños rígidos: un filtro elimina; una señal de ranking conserva alternativas y cambia su posición relativa.

3.1 De TF-IDF a BM25

Un ranking léxico necesita reconocer que no todos los términos aportan la misma evidencia. **TF-IDF** ofrece una intuición útil:

- **TF** (*term frequency*): cuántas veces aparece el término en un documento. Una aparición en el título o varias apariciones en una descripción pueden ser una señal de relación.
- **IDF** (*inverse document frequency*): qué tan raro es el término dentro de la colección. Si aparece en casi todos los documentos, discrimina poco.

El modelo **BM25** es una evolución práctica ampliamente usada en motores basados en Lucene. Sin desarrollar su fórmula, combina tres intuiciones:

1. Repetir un término aporta evidencia, pero con rendimiento decreciente: veinte apariciones no deben valer veinte veces una.
2. Los términos raros distinguen más que los comunes.
3. Un documento muy largo no debería ganar solo porque tiene más oportunidades de contener la palabra buscada.

BM25 responde bien a consultas con palabras importantes, nombres, artistas, códigos y términos que deben coincidir. No entiende por sí mismo que «música pesada» y «heavy metal» pueden expresar una intención relacionada; esa es una razón para sumar sinónimos, semántica o búsqueda híbrida.

3.2 Relevancia textual y relevancia de negocio

La puntuación textual no determina automáticamente el orden que conviene al producto. Un resultado puede ser una excelente coincidencia de palabras y no poder venderse, estar fuera de la zona buscada o ocurrir en una fecha pasada.

Una arquitectura frecuente distingue dos niveles:

- **Recuperación textual o semántica:** identifica documentos potencialmente pertinentes.
- **Ranking de negocio:** combina esa pertinencia con señales explícitas como disponibilidad, fecha, precio, reputación, distancia y reglas comerciales.

Por ejemplo, para `rock en Buenos Aires`, la pertenencia a Buenos Aires puede ser un filtro obligatorio. Entre los eventos que pasan ese filtro, la distancia desde la ubicación aproximada de la persona usuaria puede ser una señal continua: un evento cercano puede subir sin descartar todos los demás. La política debe ser explícita porque afecta visibilidad y, potencialmente, ventas.

No conviene esconder una promoción o una regla comercial dentro de una puntuación que se presenta como «más relevante». Si se incorpora una señal patrocinada, hay que definir su alcance, medir su efecto y, cuando corresponde, identificarla en la experiencia. La auditabilidad del ranking es una propiedad de producto, no solo un detalle técnico.

3.3 Medir la calidad de producto

Estas señales miden cómo percibe el resultado la persona que busca y compra; responden una pregunta distinta de la salud operativa del motor (sección 6.2) y del atraso de sincronización del índice (sección 5.3), aunque un mismo síntoma —por ejemplo, un aumento de consultas sin resultado— pueda tener causas en cualquiera de las tres capas.

Además de *precision* y *recall*, una plataforma puede observar:

- **zero-result rate:** proporción de consultas que no devuelven resultados. Un aumento puede revelar sinónimos ausentes, errores de indexación o filtros excesivos.
- **CTR de resultados:** proporción de búsquedas en las que se abre o selecciona un resultado.
- **Conversión posterior a búsqueda:** proporción de sesiones de búsqueda que culminan en una acción de valor, como reservar o comprar.
- **Latencia de búsqueda:** demora percibida hasta que aparecen resultados.

Estas señales se interpretan juntas. Un CTR alto puede esconder un ranking pobre si las personas deben probar muchos resultados. Una conversión baja puede deberse al precio, a falta de stock o a una pantalla confusa, y no necesariamente a IR. Medir permite formular hipótesis y validar cambios con experimentos controlados, no asumir que un algoritmo nuevo mejora la experiencia.

4. Experiencia de búsqueda en un catálogo

La interfaz no es una capa decorativa sobre el motor. Expresa decisiones de recuperación y permite que la persona usuaria refine su intención.

4.1 Texto, filtros y facetas

Un **filtro** impone una condición que un resultado debe cumplir. Ejemplos: eventos después de cierta fecha, productos de una categoría, un rango de precio o disponibilidad. Un filtro por ciudad puede descartar todos los documentos que no pertenecen a esa ciudad.

Una **faceta** muestra cómo se distribuyen los candidatos por una dimensión y permite explorarla: categorías, barrios, fechas o rangos de precio. Si la búsqueda devuelve 500 resultados, las facetas ayudan a responder «¿qué alternativas hay?» sin forzar a adivinar una consulta exacta.

La diferencia importa:

Mecanismo	Pregunta que responde	Efecto
Texto	«¿Qué estoy buscando?»	Recupera candidatos según términos o significado.
Filtro	«¿Qué condición es obligatoria?»	Elimina documentos que no cumplen.
Faceta	«¿Cómo puedo explorar los candidatos?»	Expone valores y conteos para refinar.
Ranking	«¿Qué debo ver primero?»	Ordena documentos que ya son elegibles.

Disponibilidad merece especial cuidado. Puede usarse como filtro de descubrimiento si el negocio no quiere mostrar productos agotados; pero la confirmación de una compra debe validar nuevamente contra el servicio dueño. El índice puede reflejar un estado anterior.

4.2 Autocomplete, typos y sinónimos

El **autocomplete** sugiere consultas, categorías o entidades mientras se escribe. Reduce esfuerzo y ayuda cuando la persona no conoce el vocabulario del catálogo. No debe confundirse con la búsqueda final: una sugerencia puede ser popular sin ser adecuada para la intención actual.

La corrección de typos y los sinónimos mejoran *recall* cuando la escritura o el vocabulario difieren entre quien busca y quien publica. Sin embargo, una expansión excesiva puede deteriorar *precision*. Un sinónimo como `rock` → `metal` puede ser razonable en ciertos contextos y erróneo en otros. Por eso los diccionarios de sinónimos deben evolucionar con evidencia de consultas, resultados vacíos y evaluación humana.

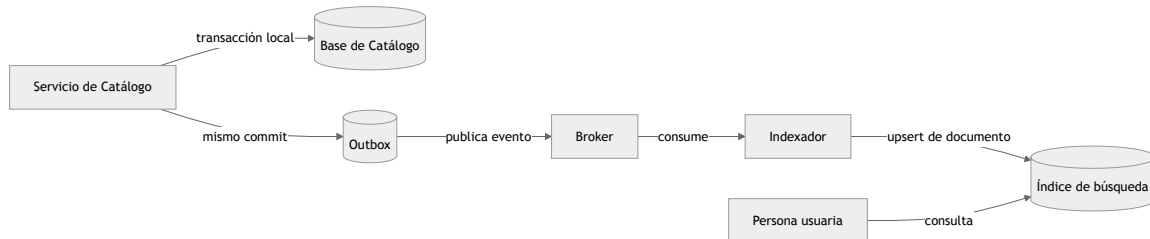
Cuando no hay resultados, la plataforma debe distinguir al menos tres causas:

1. No existe un documento que satisfaga la intención.
2. Existe, pero todavía no fue indexado o el índice está degradado.
3. Los filtros combinados hicieron imposible la consulta.

La respuesta útil no inventa resultados. Puede sugerir relajar un filtro, probar un término alternativo o informar que no hay disponibilidad; la elección depende de cuál de esas causas se conoce con suficiente confianza.

5. El índice como read model derivado

El motor de búsqueda necesita datos, pero no debe apropiarse de la responsabilidad de Catálogo. El **índice es un read model derivado**: una representación creada para una lectura particular. Usualmente está desnormalizado para evitar consultar varias bases privadas durante cada búsqueda.



El documento indexado puede reunir título, descripción, ciudad, coordenadas, fecha, rango de precio y una marca de disponibilidad. Esa copia permite recuperar, filtrar y rankear eficientemente. A cambio, introduce duplicación deliberada y consistencia eventual.

5.1 Fuente de verdad e información volátil

El documento de búsqueda es útil para elegir qué mirar, pero no es una autoridad para confirmar hechos críticos. Precio final, cupos disponibles, condiciones comerciales y permisos pueden cambiar entre la búsqueda y la acción de compra.

Una regla segura es:

Search ayuda a descubrir; el servicio dueño valida antes de producir un efecto de negocio.

Por ejemplo, una pantalla puede mostrar que quedan entradas, pero al iniciar la reserva el servicio de Inventario o Entradas debe validar la disponibilidad actual. Esa validación no se reemplaza con un campo indexado, aunque el índice se actualice rápido.

5.2 El indexador como consumidor: idempotencia y tolerancia a reentregas

Catálogo publica sus cambios mediante **Transactional Outbox**, como se vio en la clase anterior. Para el índice de búsqueda, lo que hay que resolver no es ese patrón de nuevo sino la responsabilidad que le toca al lado que consume: el indexador recibe eventos con semántica *at-least-once* y debe comportarse como un **consumidor idempotente**.

Esto significa que procesar dos veces `evento.actualizado` para la misma versión no debe generar un documento duplicado ni retroceder a un estado anterior. El indexador logra esto combinando tres elementos:

- Un **identificador estable** del documento (el mismo `id` de Catálogo), para que reprocesar el mismo evento actualice el mismo documento en vez de crear uno nuevo.
- Operaciones de **upsert**: crear el documento si no existe, reemplazarlo si ya existe, sin necesidad de verificar antes cuál es el caso.

- Una **versión monotónica** emitida por Catálogo (por ejemplo, un número de secuencia de la entidad) que el indexador compara contra la versión ya indexada antes de aplicar el cambio:

```
si evento.version > documento_indexado.version:  
    aplicar upsert(evento)  
si evento.version <= documento_indexado.version:  
    descartar (ya aplicado o desactualizado)
```

La comparación y el *upsert* deben aplicarse de forma condicional y atómica en el índice. Si un consumidor leyera una versión, decidiera que es válida y recién después escribiera en una operación separada, dos consumidores concurrentes podrían considerar válidas versiones distintas y el más lento sobrescribir el cambio más nuevo. Una versión monotónica del servicio dueño evita además depender de relojes diferentes o de timestamps empatados.

Este mecanismo evita las dos fallas típicas de una entrega *at-least-once*: duplicar un documento por una reentrega, o sobrescribir un cambio reciente con un mensaje que llegó tarde y desordenado. El detalle técnico — cómo implementa el *upsert* condicional cada motor — varía; la responsabilidad arquitectónica de tolerar duplicados y desorden es constante y no depende del motor elegido.

5.3 Consistencia eventual y freshness

A diferencia de las métricas de producto (sección 3.3) o de las señales operativas del motor (sección 6.2), esta sección trata una pregunta de diseño de datos: cuánto atraso entre la fuente y el índice es aceptable para cada campo, y qué política de negocio responde a ese atraso.

Entre el *commit* de Catálogo y la actualización del índice hay un intervalo. Durante ese tiempo:

- un alta puede no aparecer todavía;
- un evento cancelado puede seguir siendo recuperable;
- un cambio de precio o disponibilidad puede estar atrasado;
- una edición puede llegar fuera de orden si no se controla la versión.

El sistema debe definir qué **freshness** acepta para cada dato. No todos los campos merecen la misma política. Un texto descriptivo puede tolerar segundos o minutos de atraso; la disponibilidad que se muestra en una compra requiere validación en la fuente aunque el índice se actualice en pocos segundos.

La consistencia eventual no equivale a aceptar cualquier atraso. Debe medirse el tiempo de indexación, detectar eventos pendientes y tener un procedimiento para reconstruir datos. Si el atraso excede lo admisible, es un incidente de producto aunque el clúster siga respondiendo consultas.

5.4 Reindexación

Un índice puede necesitar reconstruirse si cambian los campos, el analizador de texto, un diccionario de sinónimos, la estrategia de ranking o una fuente de datos. A eso se lo llama **reindexación**.

Reindexar no es simplemente «volver a cargar». Puede requerir:

- Crear una versión nueva del índice.
- Recorrer o exportar los datos fuente para poblarla.

- Mantener actualizaciones nuevas mientras se completa la carga histórica.
- Validar cantidad, calidad y *freshness* de los documentos.
- Cambiar las consultas a la nueva versión de forma controlada.

Esto revela por qué la fuente de verdad no puede ser el índice: si se lo pierde o se modifica su modelo, debe poder reconstruirse a partir de los sistemas dueños de los datos.

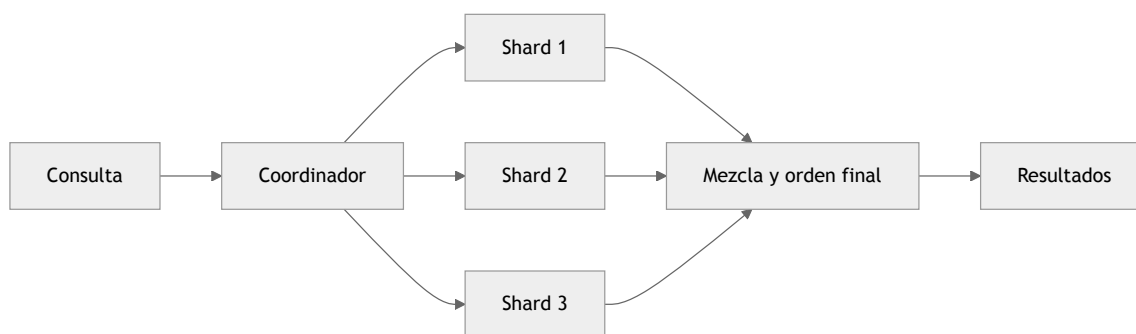
6. Escala, disponibilidad y límites operativos

Apache Lucene es una biblioteca de indexación y búsqueda. **Elasticsearch**, **OpenSearch** y **Solr** son motores que construyen capacidades de servicio sobre conceptos de Lucene; cada uno agrega opciones de distribución, APIs y operación. Para esta materia, son referencias de ecosistema, no objetos de configuración.

6.1 Shards, réplicas y query routing

Cuando el índice o la carga crecen, un motor distribuido puede dividir un índice en **shards**. Cada shard mantiene una parte de los documentos. Una consulta se distribuye a los shards pertinentes y luego se combinan los resultados parciales.

Las **réplicas** son copias de shards que ayudan a tolerar la caída de una instancia y a repartir lecturas. No hacen que todos los datos sean mágicamente consistentes: también deben recibir actualizaciones y pueden introducir costos de almacenamiento, red y coordinación.



El **query routing** decide dónde ejecutar una consulta o cómo dispersarla. Una búsqueda global suele requerir varios shards. Solo puede limitarse a un subconjunto si conoce la misma **clave de partición** usada al indexar —por ejemplo, el tenant o propietario del documento—. Elegir esa clave exige evitar particiones desbalanceadas: concentrar demasiados documentos o consultas en una sola ciudad, por ejemplo, podría producir un *hot shard*. Estos detalles importan para entender que la búsqueda distribuida tiene coordinación y costos, pero el dimensionamiento y tuning quedan fuera del alcance de esta clase.

6.2 Qué observar y qué no delegar a Search

Estas son señales de salud operativa del motor, del tipo que monitorea guardia u observabilidad: distintas de la calidad de producto medida en la sección 3.3 y de la política de freshness definida en la sección 5.3, aunque las tres compartan síntomas y deban revisarse juntas ante una anomalía.

Como mínimo, conviene detectar:

- latencia y errores de consultas;
- consultas sin resultado fuera de lo esperado;
- fallos o reintentos del indexador;
- atraso entre una modificación fuente y el documento visible;
- documentos rechazados o pendientes de reindexación.

La indisponibilidad de Search no debe impedir cobrar, reservar ni confirmar una venta. Las estrategias de timeout, circuit breaker, bulkhead, fallback y degradación de funcionalidades pertenecen al bloque de Resiliencia. La definición de SLOs, la planificación de capacidad, el despliegue y la observabilidad detallada se profundizarán en Producción y despliegue.

7. Búsqueda semántica, híbrida y RAG

La búsqueda léxica recupera términos. Es muy efectiva para nombres propios, SKU, códigos y consultas donde la palabra exacta importa. Sin embargo, puede fallar cuando la persona expresa una intención con vocabulario diferente del documento.

7.1 Embeddings y similitud semántica

Un **embedding** representa un texto como un vector numérico. Un modelo produce vectores cercanos para textos que reconoce como semánticamente relacionados. Así, una búsqueda como «música pesada» puede recuperar un documento que habla de «heavy metal» aunque no comparta exactamente todas las palabras.

La búsqueda vectorial compara el vector de la consulta con los vectores de los documentos y recupera los más similares. La similitud no es una prueba de verdad: expresa cercanía según el modelo y los datos con que fue entrenado. Puede recuperar un documento temáticamente parecido pero que no satisface una condición específica, como fecha, ubicación o disponibilidad.

Por eso no conviene reemplazar automáticamente toda búsqueda léxica por vectores. Para `SKU-8432`, un número de pedido, un nombre de artista o una restricción legal, una coincidencia exacta suele ser más confiable, más explicable y a menudo más barata.

7.2 Búsqueda híbrida

La **búsqueda híbrida** combina recuperación léxica —por ejemplo BM25— con recuperación vectorial. Cada mecanismo ofrece una señal distinta:

Mecanismo	Suele aportar	Puede fallar en
Léxico / BM25	Términos exactos, nombres, códigos y coincidencias explicables	Sinónimos o intención expresada con otras palabras
Vectorial	Significado, paráfrasis y vocabulario alternativo	Identificadores exactos, restricciones duras y similitudes engañosas
Híbrido	Cobertura de ambos tipos de intención	Mayor costo, complejidad y necesidad de calibrar señales

Combinar no significa sumar puntuaciones sin criterio. Los puntajes de sistemas diferentes pueden tener escalas incompatibles. Se necesita una política de fusión, evaluación con consultas representativas y límites de latencia. Los filtros de permisos, fecha o ciudad deben mantenerse como condiciones explícitas; una alta similitud semántica no autoriza a ignorarlos.

7.3 RAG: generación basada en recuperación

RAG (*Retrieval-Augmented Generation*) agrega una etapa de recuperación antes de solicitar una respuesta a un modelo de lenguaje:



En vez de pedir al modelo que responda solo desde su conocimiento previo, se recuperan fragmentos relevantes y se los entrega como contexto. Esto puede fundamentar una respuesta sobre políticas, descripciones o condiciones documentadas de la plataforma.

RAG reduce una fuente frecuente de respuestas inventadas, pero no garantiza que la respuesta sea correcta. Puede fallar porque:

- recuperó fragmentos incorrectos o incompletos;
- los documentos estaban desactualizados;
- la consulta no aplicó permisos antes de recuperar;
- el texto se fragmentó de forma que perdió contexto;
- el modelo interpretó o sintetizó mal incluso un contexto correcto.

Un diseño responsable define tamaño y límites de los fragmentos, metadatos, filtros por permisos, versiones de contenido, referencias que la persona usuaria pueda consultar y evaluación separada de recuperación y generación.

Los datos volátiles requieren un límite adicional. Un RAG puede explicar las condiciones de un evento a partir de documentos indexados, pero no debe afirmar que hay entradas disponibles o que un precio sigue vigente sin validar una fuente actualizada. Para efectuar una reserva o confirmar una compra, la validación sigue perteneciendo al servicio dueño.

8. Casos reales

8.1 Wikimedia: una plataforma de contenido necesita un índice separado

Wikimedia implementa su búsqueda mediante CirrusSearch, una extensión de MediaWiki que indexa el contenido de las páginas en un backend de búsqueda. El caso permite observar dos ideas centrales: el contenido de las páginas y el índice cumplen funciones diferentes, y el índice debe actualizarse cuando cambia una página. La documentación de CirrusSearch describe explícitamente su objetivo de mantener un índice de búsqueda *near-real-time* para páginas de wiki y el proyecto está migrando su backend hacia OpenSearch. [Documentación de CirrusSearch](#).

La lección no es copiar su tecnología. Es reconocer que, cuando la experiencia depende de recuperar texto relevante en una colección grande, el índice se convierte en una vista especializada y reconstruible, no en la única copia de la información.

8.2 Netflix: búsqueda híbrida para encontrar escenas dentro del contenido

Los equipos de producción y edición de Netflix necesitan encontrar momentos específicos dentro de horas de material audiovisual —una escena, un objeto, una línea de diálogo—, algo que una búsqueda por metadatos de texto exactos no siempre resuelve. El Netflix Technology Blog describe un sistema de búsqueda de video que combina coincidencias textuales (diálogo, metadatos, descripciones) con embeddings multimodales que representan audio, video y texto, de forma que una consulta pueda recuperar tanto coincidencias literales como escenas semánticamente relacionadas aunque no compartan las mismas palabras. [Netflix TechBlog — Building In-Video Search](#); [Netflix TechBlog — Synchronizing the Senses: Powering Multimodal Intelligence for Video Search](#).

El caso muestra, con una empresa real y no con documentación de un producto, por qué combinar señal léxica y semántica no es una característica accesorio de un motor sino una respuesta a una necesidad concreta: encontrar contenido cuando la palabra exacta no alcanza.

8.3 Uber: indexar el espacio para resolver proximidad a escala

Uber desarrolló y liberó como código abierto **H3**, un sistema de indexación geoespacial que agrupa ubicaciones en celdas hexagonales jerárquicas. Esto permite aproximar consultas de proximidad sobre áreas indexadas, en vez de calcular la distancia contra cada punto del mapa. Uber lo usa para analizar y optimizar decisiones de su marketplace, como despacho y precios. [Uber Blog — H3: Uber's Hexagonal Hierarchical Spatial Index](#).

El caso no presenta H3 como una implementación de búsqueda textual ni como una dependencia obligatoria de Search. Muestra que «cerca» puede requerir una estructura de indexación propia: en una plataforma con muchos documentos ubicados en el espacio, la proximidad geográfica es una señal calculable y no un simple filtro por ciudad.

9. Caso aplicado: búsqueda híbrida para una plataforma de venta

La plataforma necesita responder la consulta `rock en Buenos Aires`.

9.1 Datos y actualización

Catálogo es dueño de cada evento. Cuando se crea o modifica uno, confirma su cambio junto con un registro Outbox. Un publicador entrega `evento.actualizado` a un broker y el Indexador realiza un *upsert* en Search. El documento contiene campos de lectura: título, descripción, artista, fecha, ciudad, coordenadas aproximadas, precio desde, categorías y estado de publicación.

Si el evento cambia de fecha, el Indexador recibe una versión posterior y reemplaza el documento correspondiente. Si el mensaje se reentrega, el mismo *upsert* no crea un duplicado. Si llega un mensaje antiguo después de uno nuevo, la versión impide volver al estado anterior.

9.2 Consulta y orden

La plataforma procesa la consulta en esta secuencia:

1. Normaliza `rock en Buenos Aires` y aplica sinónimos cuidadosamente evaluados.
2. Recupera candidatos con BM25 sobre título, artista y descripción, y con una señal semántica para detectar vocabulario equivalente.
3. Filtra por ciudad y fecha solicitadas. Si la persona exige accesibilidad o un rango de precio, esas condiciones también son filtros explícitos.
4. Ordena los resultados elegibles combinando pertinencia, cercanía geográfica, disponibilidad indexada y reglas de negocio transparentes.
5. Muestra los resultados con atributos de lectura; al iniciar una compra vuelve a validar disponibilidad y precio en los servicios dueños.

El orden de esas operaciones no es un detalle menor. Si se deja que una señal vectorial reemplace un filtro, podría aparecer un evento semánticamente parecido pero fuera de la ciudad. Si se toma el precio indexado como definitivo, una compra puede confirmar un importe desactualizado.

9.3 Asistente RAG asociado

Sobre los eventos recuperados, un asistente puede responder «¿qué artistas participan?» o «¿qué condiciones de ingreso tiene este festival?». Para hacerlo, el *retriever* debe limitarse a documentos autorizados y devolver referencias. Si la pregunta es «¿todavía quedan entradas?» o «¿cuál es el precio final?», el asistente no debe responder desde el índice como si fuera una fuente transaccional: debe informar que necesita una validación actual o consultar el servicio dueño mediante un mecanismo controlado.

10. Trade-offs y decisiones arquitectónicas

Decisión	Gana	Paga	Cuándo conviene
Consultar la base transaccional	Menos componentes y consistencia directa	Poco ranking, menor capacidad de descubrimiento y carga compartida	Catálogo pequeño y búsquedas exactas
Índice léxico separado	Recuperación rápida y ranking textual explicable	Duplicación, sincronización y consistencia eventual	Texto, filtros y volumen justifican una capacidad especializada
Ranking con señales de negocio	Resultados más útiles para el producto	Política menos neutral; requiere medición y auditoría	La pertinencia textual por sí sola no satisface la decisión de compra
Búsqueda vectorial	Encuentra intención y vocabulario alternativo	Costos, latencia y resultados semánticamente cercanos pero incorrectos	Consultas abiertas sobre lenguaje natural
Búsqueda híbrida	Combina exactitud léxica y cobertura semántica	Calibración de señales y mayor complejidad	Conviven códigos/nombres exactos e intención expresada libremente
RAG	Respuestas fundamentadas en documentos recuperados	Depende de retrieval, permisos y freshness; no valida hechos transaccionales	Preguntas sobre conocimiento documentado, no confirmaciones de negocio

11. Puntos clave

- Una búsqueda de descubrimiento no es una consulta por clave: debe recuperar y ordenar documentos relevantes.
- El índice invertido permite encontrar candidatos por términos sin recorrer toda la colección.
- *Precision* mide cuán pertinentes son los resultados devueltos; *recall*, cuántos de los pertinentes fueron recuperados.
- BM25 rankea coincidencias léxicas considerando frecuencia, rareza y longitud del documento.
- Sinónimos expanden una consulta; filtros eliminan candidatos; el ranking ordena los que siguen siendo elegibles.
- La relevancia textual y la relevancia de negocio son señales distintas que deben combinarse de forma explícita y medible.

- El índice es un read model derivado: Search ayuda a descubrir, pero Catálogo u otro servicio dueño valida las operaciones críticas.
- Transactional Outbox permite propagar cambios de manera confiable, pero el indexador debe tolerar reentregas y mensajes tardíos.
- La consistencia eventual exige definir y medir el freshness aceptable para cada tipo de dato.
- La búsqueda híbrida combina recuperación léxica y semántica; no todos los problemas mejoran con embeddings.
- RAG recupera contexto antes de generar una respuesta, pero una mala recuperación, permisos incorrectos o contenido desactualizado siguen produciendo respuestas no confiables.

12. Referencias

- [MediaWiki — CirrusSearch.](#)
- [Wikitech — CirrusSearch.](#)
- [Elastic — Elasticsearch Reference.](#)
- [Elastic — Distributed architecture.](#)
- [AWS — Neural Search and Hybrid Search en OpenSearch.](#)
- [AWS — Vector search collections en OpenSearch.](#)
- [AWS Prescriptive Guidance — Retrievers para RAG.](#)
- [Netflix TechBlog — Building In-Video Search.](#)
- [Netflix TechBlog — Synchronizing the Senses: Powering Multimodal Intelligence for Video Search.](#)
- [Uber Blog — H3: Uber's Hexagonal Hierarchical Spatial Index.](#)